

Agentic Search 2026 与 MR-Search 汇报逐字稿

适用场景：约 18-22 分钟汇报

目标：不是逐字复述 PPT，而是围绕训练过程、工程卡点、指标诊断、reward 设计和项目 insight，把 MR-Search case 讲成一个完整故事。

第 1 页：Agentic Search 2026

大家好，我今天分享的主题是 **Agentic Search 2026 与 MRSearcher**。

这页我想先定一个大的背景：过去我们讲搜索增强，一般是在讲 RAG，也就是文档切块、向量化、召回、然后把 top-k 交给模型回答。但到 2026 年，搜索这件事本身正在从一个“外部模块”，变成模型推理过程中的一种 **policy**。

也就是说，模型不再只是被动接收检索结果，而是要学会：什么时候需要搜，搜什么，搜几次，拿到结果之后要不要继续搜，以及什么时候停止并回答。

所以今天这份汇报不是单纯讲某篇论文，也不是单纯讲一个 RAG 系统，而是想回答一个问题：**搜索能不能被训练？搜索能不能像推理策略一样被优化？**

后面我会先讲从传统 RAG 到 Agentic RAG，再讲 Search-R1 和 MRSearch 的方法，然后落到我自己的运维 MR-Search case。这个 case 的重点不是“模型多搜了几次”，而是我们怎么把搜索过程做成一个可以观测、可以诊断、可以回流、可以持续优化的系统。

第 2 页：传统 RAG 的基线

这一页讲的是传统 RAG 的基础形态：**Chunk – Embed – Retrieve – Generate**。

传统 RAG 的核心贡献是把企业内部文档变成可检索资产。比如 PDF、网页、Wiki，经过 OCR、版面解析、表格识别、切块，然后进入向量数据库或者 BM25 / Elasticsearch，最后模型基于 top-k 文档生成答案。

但它的问题也很明显：传统 RAG 的模型通常只负责最后一步生成。换句话说，搜索策略大部分是系统预设的，比如固定 top-k、固定 rerank、固定上下文拼接方式。

这对于单跳问题是够的，比如“某个状态码是什么意思”“某个配置项怎么设置”。但一旦问题变成多跳，比如“昨晚支付网关为什么超时”，传统 RAG 就会吃力。因为这个问题不是搜一篇文档就能回答，它需要先定位服务实体，再查告警，再查变更，再查历史 SOP，最后综合证据。

所以传统 RAG 的天花板不在于它不能检索，而在于它没有把“搜索路径”建模成一个可以学习和优化的过程。

这一页的核心结论是：**2025 年的主线是把文档变成可检索资产；但 2026 年的主线会变成让模型学会如何使用这些资产。**

第 3 页：Agentic Search 六条路线

这一页我想把 Agentic Search 的方向拆成六条线。

这六条不是并列堆概念，而是在同一条搜索链路上协同工作。

第一条是基础的 **Search-Enhanced Reasoning**，也就是推理过程中用搜索补知识。

第二条是 **Retrieval-Reasoning Coupling**，它关注检索和推理怎么联合建模，不再是先检索后回答的简单串联。

第三条是 **RL Search Policy**，也就是今天重点讲的方向：把搜索动作放进 rollout，让模型学会搜索策略。

第四条是 **Deep Research**，它更像长程任务规划，会做任务拆解、多轮搜索、证据整理和报告生成。

第五条是 **Search Scaling**，包括并行搜索、树搜索、多智能体协作，本质是扩大探索空间。

第六条是 **Memory-Driven Search**，把历史事实、搜索轨迹、失败经验保存下来，支撑下一轮搜索。

这里我想强调一个判断：Agentic Search 的核心不是“多加几个工具”，而是把搜索变成一个闭环系统。输入是问题，输出不是单次检索结果，而是一条可解释的搜索轨迹。

这也是后面 MR-Search 的基本思想：模型要学的不只是 answer，而是 search-reason-answer loop。

第 4 页：Agentic RAG / DeepRead

这一页讲 Agentic RAG，PPT 里用的是 DeepRead 的例子。

它和传统 RAG 最大的区别是：传统 RAG 通常是一次检索，然后回答；Agentic RAG 会把文档先做层级索引，然后让模型在多轮 trajectory 里决定下一步是 retrieve 还是 read。

比如一个文档很长，模型不是一开始就把所有内容塞进上下文，而是先检索相关章节，再读某个段落，再根据读到的内容决定是否继续检索。

所以它的控制流大概是：

Question → Retrieve → Read → Retrieve → Read → Answer

这已经比传统 RAG 前进了一步，因为模型开始参与工具调用轨迹，而不只是最后生成答案。

但这里也埋下了后面的问题：Agentic RAG 主要靠 prompt 和规则驱动。它可以“会调用工具”，但不代表它能稳定地学会搜索策略。比如它这次搜错了，下次遇到类似问题，如果没有训练或者回流机制，它不一定会变好。

所以这一页要引出后面的 Search-R1：**Agentic RAG 让搜索过程显式化，但 Search-RL 进一步让这个过程可训练。**

第 5 页：Search-R1 的核心机制

这一页是第一个重点：Search-R1。

Search-R1 的关键变化是把搜索动作写进模型 rollout。也就是说，搜索不再是系统在外面偷偷做的，而是变成模型生成轨迹的一部分。

格式上可以理解成：

```
<think>当前推理</think>
<search>搜索请求</search>
<information>搜索系统返回的结果</information>
<think>继续推理</think>
<answer>最终答案</answer>
```

这里有两个很关键的点。

第一，<search> 是模型生成的 action。模型需要决定当前要不要搜，以及搜什么。

第二，<information> 不是模型生成的，而是搜索环境返回的 observation。所以训练时这部分要 mask 掉，不能让模型因为不是自己生成的 token 被奖励或惩罚。

PPT 里举了 Curious 香水的例子。普通模型可能把 singer 误判成 Beyonce，但 Search-R1 可以先搜 Curious fragrance information，发现它和 Britney Spears 有关，再搜 Britney Spears birthplace，最后得到 McComb, Mississippi。

这个例子说明，Search-R1 的价值不是模型记住了答案，而是它通过多轮搜索修正了实体链。

换到企业运维场景也是一样。用户问“支付网关昨晚为什么超时”，模型不能凭常识回答，它需要查“支付网关对应哪个服务”“昨晚有没有告警”“有没有变更”“SOP 怎么处理”。这就是搜索策略。

这一页的核心 insight 是：**Search-R1 把搜索从一个外部插件，变成了 policy rollout 的一部分。**

第 6 页：MRSearch：从单次搜索到多轮自反

这一页讲 MRSearch，它是在 Search-R1 之上继续往前走。

Search-R1 更多是在一个 episode 里做搜索、推理、回答。但如果这个 episode 里实体链已经走错，最后 reward 很稀疏，模型很难知道到底哪里该重搜。

MRSearch 的思路是把失败答案变成下一轮 episode 的输入。也就是：第一轮回答错了，不是直接结束，而是做 self-reflection，检查证据缺口和实体链风险，再追加搜索，最后生成更好的答案。

这里可以理解成：

Episode 1 : Reason → Toolcall → ToolObs → Answer

Self-Reflection : 检查哪里缺证据，哪里实体可能错

Additional Search : 围绕关键实体、约束条件补充检索

Episode N : 把上一轮失败轨迹作为上下文，生成更准的 toolcall 和 answer

这对训练很重要，因为它把“答错后的补救行为”也纳入了学习轨迹。也就是说，模型不仅学会第一次怎么搜，还学会在答案不确定、证据不足、实体链有风险时怎么修正。

我这里想强调的是：MRSearch 不是简单地多搜索一次，而是把失败轨迹制度化。它让失败不只是 loss，而是下一轮搜索策略的监督信号。

第 7 页：为什么运维场景适合 Search Agent

这一页进入我们的企业运维 case。

运维场景非常适合 Search Agent，因为它的信息很多，但答案很难直接拼出来。

一线工程师每天面对的是告警、日志、指标、Wiki、SOP、工单、变更记录。一个问题背后会牵涉服务、租户、版本、配置项、错误码和依赖链路。

比如“昨晚支付网关为什么超时”，这个问题不是一个普通 FAQ。它需要连续排查：

- 先确认“支付网关”到底对应哪个服务；
- 再查昨晚这个服务有没有告警；
- 再查同时间段是否有变更；
- 再看历史工单里有没有类似案例；
- 最后结合 SOP 给出处理建议。

运维知识库还有几个典型问题：

第一，资料碎片化。文档散在不同系统里，而且同一个服务可能有多个名字，比如支付网关、PGW、payment-gateway。

第二，时效和权限敏感。旧 SOP 和新 SOP 不能混用，不同租户的文档也不能跨权限访问。

第三，天然多跳。运维排障不是一次搜索，而是现象到服务、服务到告警、告警到变更、变更到根因。

所以这个场景里，Agent 要做的不是“写一段看起来专业的回答”，而是要主动查资料、对齐实体、交叉验证证据，并给出可执行建议。

这一页的核心结论是：**运维 Agent 的难点不在回答，而在连续搜索和证据拼接。**

第 8 页：为什么不直接用 Agentic RAG

这一页回答一个甲方常问的问题：为什么不直接用 Agentic RAG，为什么还要 RL？

我的回答是：我们不是不用 Agentic RAG，而是先用它做 baseline。

Agentic RAG 能把流程跑通，也能解决一部分单跳问题。比如查一个 SOP、查一个配置项、查某个状态码，这些问题 Agentic RAG 已经可以做得不错。

但复杂运维多跳里，问题不只是“会不会调用工具”，而是“搜索策略稳不稳”。

比如同一个问题，模型这次可能先搜 SOP，下次可能先搜告警，再下次可能直接回答。搜什么、搜几次、什么时候停，主要靠 prompt 和规则。它这次搜错了，下次不一定会变好。

RL 的投入点就在这里：当检索地基已经修好之后，我们希望利用 rollout 里的成功和失败轨迹，把有效搜索路径强化，把重复搜索、过早停止、命中证据但答错这些行为压下去。

所以这页我想表达的不是“Agentic RAG 不行”，而是：Agentic RAG 是第一阶段，它让工具调用显式化；RL 是第二阶段，它让搜索策略可优化。

一句话：**Agentic RAG 解决能不能跑，Search-RL 解决能不能稳定变好。**

第 9 页：Badcase 先分诊，再决定训什么

这一页是整个项目方法论里最重要的一页之一。

很多人做 RL 会有一个误区：看到 badcase 就想塞回训练。这个是不对的。

因为 badcase 可能坏在不同层：Query、Retrieval、Context、Answer、Loop，也可能根本是 Infra 问题。

所以我们先做 badcase router，先问“坏在哪一层”，再问“RL 能不能学”。

比如 Query Too Broad，就是 raw query 太长，污染 BM25。这个时候应该看 raw_search_query、cleaned_query、first_hop_query，然后回流到 single-rewrite v2，而不是直接训练模型回答。

No Hit Wrong 是没搜到证据还答错。这类问题优先修索引、别名、ranking。因为模型没有看到证据，不能怪它答错。

Hit But Wrong 是搜到了但没用对。这才更适合回流到 formatter、answer extraction 或 reward，因为正确证据已经出现，但模型没有转成正确答案。

Final Worse 是 candidate 已经对了，但 final 阶段改坏。这个要回流 stop gate 和 answer preservation。

Over Search 是重复搜、顶预算，这类优先用 hard gate 控制。

Infra Badcase，比如端口挂了、LR 是旧的、环境变量没进 worker，这类根本不能进入训练，先修工程。

这一页的核心 insight 是：**badcase 不是错题集，而是路由表。它告诉我们下一轮应该改哪个模块。**

第 10 页：训练数据流 + 工具控制流

这一页讲整个训练系统怎么跑。

左边是训练数据流，右边是工具控制流。

训练数据流可以这样讲：

第一步是题库包。样本里不只有 question 和 answer，还包括标准答案、别名和证据监督信号。

第二步是多轨迹展开。每个问题会生成多条 search / evidence / answer 轨迹，比如每题 8 条 rollout。这样我们能看见模型是否偶尔能搜对。

第三步是搜索环境。模型生成 <search>，检索池返回候选证据，这些结果进入轨迹上下文。

第四步是证据注入。标题、片段、命中窗口被压成模型可读的 <information>。

第五步是评分回传。最终答案质量和过程信号一起打分，回传给 PPO / GRPO 更新 policy。

右边的工具控制流是另一条线。

模型本身只生成搜索意图，不直接访问数据库。Tool controller 会拦截 <search>，检查空 query、重复 query、搜索预算和 hard gate。然后 query 会经过 rewrite，从原始长句清洗成更稳的 first-hop query。再路由到 insight / synthetic-data 的检索端口，返回 top3 / pack50 的证据包。最后 stop gate 控制继续搜还是回答。

这一页要强调一个工程判断：**模型不是自由地调用所有东西，而是在一个可审计、可拦截、可记录的工具环境里行动。**

第 11 页：6 工程坑

这一页我建议你讲得稍微具体一点，因为它能体现项目真实性。

这 6 个坑分成系统工程坑和 Agent RL 机制坑。

第一个系统坑是知识库数据脏。运维知识库服务别名不统一，旧 SOP 和新 SOP 混杂，重复文档很多，固定长度切块会把一个完整处置流程切断。这会导致正确答案在库里，但进不了 context。

第二个系统坑是 retrieval 是环境。检索服务一挂，rollout 里的 information 就为空，reward 信号直接被污染。所以要做 FTS5/BM25 health gate，检查 topk=3、pack50、fusion10 等实际配置。

第三个系统坑是多机配置会失真。Ray worker 环境变量不一定继承 wrapper，resume checkpoint 可能把旧 LR 带回来，quick32 也可能误导实验方向。所以必须强制看 actor/lr、resume path、quick32 / quick128。

然后是三个策略坑。

第四个坑是 search action 不等于好 query。raw query 太长会污染 BM25，rewrite 太狠又会裁掉多跳约束。所以我们用 single-rewrite v2 加 raw fallback 的思路。

第五个坑是 hit 不等于答对。retrieval_alias_hit=true 也可能 final 错；candidate 对了，final 也可能改坏。所以要区分 hit-correct、hit-wrong、final-worse。

第六个坑是 loop / stop / reward。多搜、少搜、重复搜都会错。强奖励少搜会过早回答，强奖励 hit 会搜到不用。所以 hard gate 先兜底，reward 以 outcome 主导、process 轻约束。

这页最后可以收束成一句：**系统工程坑解决“证据能不能稳定给到模型”，Agent RL 机制坑解决“模型能不能稳定搜对、用对、停对”。**

第 12 页：Reward 设计和反 hacking

这一页讲 reward。

我们的 reward 设计不是奖励“多搜”，也不是奖励“少搜”，而是奖励“搜对以后用对”。

整体分三层：

第一是 outcome reward，也就是 final EM / F1。最终答案对不对是唯一主目标。

第二是 process reward，但只是轻约束，比如 hit_correct 加分，hit_wrong、unsupported、repeat、final_worse 轻扣。

第三是 hard gate。重复 query、搜索预算、命中后回答这些明显系统约束，不完全交给 reward 学，而是用机制层先挡住。

为什么要这样设计？因为 reward 非常容易被 hack。

第一类 hacking 是少搜骗分。如果 search penalty 太强，模型会学会少搜、早点 answer。表面上 finished 上升，但 hit 和 F1 下降。

第二类是单纯 hit 骗分。模型搜到了相关词，但 final answer 还是错。所以不能单纯奖励 retrieval hit，而要 hit + correct 才加分，hit + wrong 反而扣分。

第三类是无证据回答。模型用很专业的运维口吻回答，但 selected evidence 根本不支持 final。这类要 unsupported penalty。

第四类是 retry 改坏。candidate 已经正确，但 reflection 越多，引入噪声，final 反而错。这类用 final-worse 约束，证据足够时就 answer。

所以这页的核心 insight 是：**process reward 只能防坏行为，不能反过来定义目标。目标仍然是 final answer correctness。**

第 13 页：如何看模型有没有潜在正确路径

这一页非常适合讲 RL 为什么值得做。

我们不应该只看 pass@1。pass@1 是一次采样能不能答对，代表线上默认能力。但 RL 之前更关键的是看 pass@k：多采样 k 次里，有没有至少一条正确轨迹。

如果 pass@1 是 0.35，pass@8 是 0.62，说明模型不是不会，而是不稳定。它偶尔能搜对，但默认策略不稳定。这种情况适合 RL。

如果 pass@1 是 0.30，pass@8 是 0.34，说明多采样也找不到正确路径。这个时候不要急着训，先修 query、retrieval、别名和索引。

如果 pass@1 已经 0.70，pass@8 只是 0.74，说明策略已经比较稳定，继续 RL 边际收益可能有限。

然后我们还要把潜在路径拆到五层看。

Query 层看 raw / cleaned / first-hop，8 条里有没有至少一条生成正确 first-hop query。

Retrieval 层看 retrieval_alias_hit / query_gold_hit，是否能搜到 gold alias 或 gold doc。

Context 层看 selected_evidence / clip_ratio，gold 是否真正进入模型可见证据。

Answer 层看 candidate / final，如果 candidate 已经对但 final 改错，就说明 evidence use 有潜力，但 final 不稳。

Loop 层看 search_count / final_worse_after_retry，如果第 2 轮已对、第 3 轮继续搜后变差，说明 stop policy 有问题。

训练后我们希望看到：pass@1 上升，pass@8 不一定大涨，但 pass@8 - pass@1 的差距缩小。同时 hit-but-wrong、final-worse、repeated-query、unsupported answer 下降。

这页的核心观点是：**pass@k 看潜力，pass@1 看线上能力；RL 的价值就是把 pass@k 里的好轨迹压进 pass@1。**

第 14 页：Trace 自循环飞轮

这一页讲项目真正的工程资产：trace 飞轮。

每个 checkpoint 之后，我们不只看一个榜单，而是产出 summary metrics 和 per-sample trace。

trace 至少保存 qid、checkpoint、raw / cleaned / first-hop query、top3 evidence title、selected evidence、candidate answer、final answer、gold alias、search count、hard blocks 和归因标签。

然后做指标分桶：no-hit、hit-but-wrong、final-worse、repeat、unsupported。

每个桶抽 20 到 50 个 badcase，人工看是不是同一根因集中出现。

之后分流回流：Query 问题回流到 rewrite；retrieval 问题回流到 ranking 和别名表；context miss 回流到 hit window、bridge clue、evidence pack；hit-but-wrong 回流到 formatter、answer extraction 和 hit_wrong penalty；final-worse 回流到 stop gate 和 answer preservation；reward hack 回流到 reward 权重和 verifier；infra 问题回流 SOP 和 health check。

接下来做小步实验。一次只改一个变量，先 quick32，再 quick128。只有目标桶改善、主指标不下降、没有新 hacking，才进入长训。

然后新模型继续产 trace，进入下一轮循环。

这一页要讲出一个核心 insight：**训练不是一次性实验，而是一个可自循环的诊断系统。指标告诉哪里坏，trace 解释为什么坏，badcase 决定下一轮改什么。**

第 15 页：怎么证明真的变好了

这一页讲观测仪表盘。

我们不能只看某个指标涨了，比如 final F1 涨一点，或者 search_count 降一点。真正要看的是好轨迹有没有从偶发采样变成默认策略。

这里有三个核心观测。

第一，Potential Path gap，也就是 pass@8 - pass@1 是否缩小。如果缩小，同时 pass@1 上升，说明以前偶尔对的路径正在变成默认路径。

第二，Evidence Use。retrieval hit 不能停在上限，而要稳定转成 final correctness。也就是 hit - final 的链路要变强。

第三，Loop Health。final-worse、repeat、unsupported 要一起下降，才说明不是 reward hack。

这页里列了几个观测签名。

如果 pass@1 上升、gap 缩小、目标桶 badcase 下降，说明 RL 把潜在路径稳定化，可以放大样本进入长训。

如果 hit 上升但 final F1 不升，甚至 hit-but-wrong 上升，说明证据进来了，但模型没用好，应该修 formatter / answer。

如果 search_count 下降但 no-hit-wrong 上升，说明模型可能学会了早停，要调 stop / reward。

如果 unsupported、final-worse、repeat 都上升，说明出现 reward hacking 或 loop 破坏，要停训回滚，加 hard gate。

这页最后一句很重要：F1 / hit / wrong / repeat / stop 必须一起看。

换句话说，**模型是不是变好了，不看单点涨跌，而看整条 search-reason-answer 链路是否更健康。**

第 16 页：最终总结

最后一页是总结页。

这里我建议你不要再展开太多机制，而是把整个项目收束成一句话：

MR-Search 的价值不是让模型多搜，而是让搜索变得可诊断、可控制、可迭代。

什么时候值得做？

当 pass@k 高于 pass@1，说明模型偶尔能搜对；当 badcase 能被 trace 归因，说明这些好轨迹可以被强化。这时候 RL 有价值。

什么时候先别做？

如果 pass@k 也低，说明正确路径还没出现。这个时候应该先修数据、query、retrieval、verifier 和 quick eval，不要直接长训。因为 RL 没有正确轨迹可学。

长期价值是什么？

不是某个 checkpoint，而是一套 Search 运营系统：trace 记录、badcase 分桶、模块回流、quick gate 验证，让每次失败都能进入下一轮优化。

我最后想强调的是：Search-RL 项目真正难的不是“模型能不能调工具”，而是能不能把搜索过程做成一个工程闭环。只有当 query、retrieval、context、answer、reward、loop 都可观察、可归因、可回流，搜索才真的从一个功能变成一种可运营能力。

所以这份项目最终沉淀的核心 insight 是：

Make search operable. 让搜索可运营。

结尾备用总结

整场汇报如果用一段话收尾，可以这样说：

今天这套 MR-Search case 给我的最大启发是，Agentic Search 不是简单地让模型多调用几次搜索工具。真正有价值的是把搜索过程变成一条可训练、可诊断、可回流的链路。传统 RAG 解决的是“文档能不能被搜到”，Search-RL 要解决的是“模型能不能稳定地搜对、用

对、停对”。所以我们最终不是只追 EM/F1，而是通过 trace 去看 query、retrieval、context、answer、loop 每一层怎么变化；不是所有 badcase 都丢给 RL，而是先分桶归因；不是看到 quick32 涨一点就长训，而是用 quick128 和目标桶指标做 gate。这个项目真正留下的不是某个模型，而是一套让搜索系统持续变好的工程飞轮。